

# SMS SPAM DETECTION USING MACHINE LEARNING

## INTERNSHIP PROJECT REPORT

Submitted by

**Mandeep Kaur**

Submitted To

**Naviotech Solution Pvt Ltd**

Academic Session

**2025–2026**

---

### CERTIFICATE

This is to certify that **Mandeep Kaur** has successfully completed the project titled "**SMS Spam Detection Using Machine Learning**" as part of the internship program at Naviotech Solution. The project was carried out under guidance and demonstrates practical implementation of Machine Learning techniques for spam message classification.

---

### ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Naviotech Solution for providing me with the opportunity to work on this project. This internship helped me gain practical knowledge of Machine Learning, Python programming, data preprocessing, and model development.

I am also thankful to my mentors and trainers for their valuable guidance and support throughout the project. Their continuous encouragement enabled me to successfully complete this work.

## TABLE OF CONTENTS

1. Introduction
2. Project Overview
3. Problem Statement
4. Objectives
5. Scope of the Project
6. Literature Review
7. Dataset Description
8. Methodology
9. Data Preprocessing
10. Feature Extraction
11. Machine Learning Algorithm
12. Model Training
13. Model Testing
14. Results and Analysis
15. Applications
16. Advantages
17. Limitations
18. Future Scope
19. Conclusion
20. References

---

## 1. INTRODUCTION

In today's digital era, mobile communication plays an important role in daily life. Millions of text messages are exchanged every day across the world. Along with useful communication, unwanted and fraudulent messages known as spam messages have become a significant problem.

Spam messages often contain advertisements, promotional offers, phishing links, or fraudulent content. These messages can waste users' time and sometimes lead to security risks. Therefore, an intelligent system capable of automatically identifying spam messages is highly beneficial.

Machine Learning provides effective techniques for classifying messages based on their content. This project develops an SMS Spam Detection System using Machine Learning and Natural Language Processing techniques.

---

## 2. PROJECT OVERVIEW

The SMS Spam Detection System is designed to classify incoming text messages into two categories:

- Spam Messages
- Ham (Legitimate) Messages

The system analyzes the textual content of messages and predicts whether a message is spam or not. Logistic Regression and TF-IDF Vectorization are used to build the classification model.

---

## 3. PROBLEM STATEMENT

The rapid increase in spam messages creates inconvenience for users and poses security risks. Manual filtering is inefficient and time-consuming.

Therefore, there is a need for an automated system that can accurately classify SMS messages and protect users from unwanted communications.

---

## 4. OBJECTIVES

The main objectives of this project are:

- To develop a machine learning-based SMS spam detection system.
  - To preprocess and clean text data.
  - To convert text into numerical features using TF-IDF.
  - To train a Logistic Regression model.
  - To evaluate model performance using accuracy metrics.
  - To classify new SMS messages automatically.
-

## 5. SCOPE OF THE PROJECT

The project can be applied in:

- Mobile messaging applications
- Email filtering systems
- Fraud detection systems
- Customer communication platforms
- Cybersecurity applications

The developed model can be integrated into real-world communication systems to improve message security.

---

## 6. LITERATURE REVIEW

Researchers have extensively used Machine Learning techniques for spam detection. Various algorithms such as Naive Bayes, Support Vector Machines, Random Forest, and Logistic Regression have been applied for message classification.

Recent studies indicate that TF-IDF combined with Logistic Regression provides high accuracy and efficient performance in text classification tasks.

## 7. DATASET DESCRIPTION

The project uses the SMS Spam Collection Dataset.

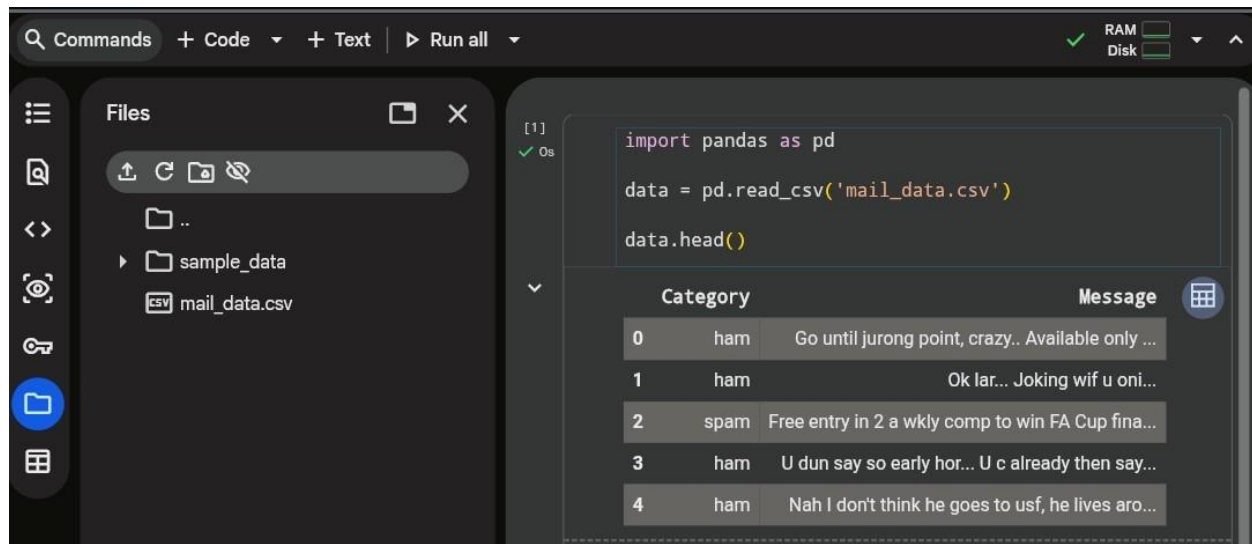
### Dataset Features

| Feature | Description |
|---------|-------------|
|---------|-------------|

|          |             |
|----------|-------------|
| Category | Spam or Ham |
|----------|-------------|

|         |             |
|---------|-------------|
| Message | SMS Content |
|---------|-------------|

## Dataset Statistics



The screenshot shows a Jupyter Notebook environment. The top bar includes a search bar, tabs for 'Commands', '+ Code', '+ Text', and a 'Run all' button. On the right, RAM and Disk usage are displayed. The left sidebar shows a file explorer with a folder named 'sample\_data' and a file named 'mail\_data.csv'. The main area contains a code cell with the following Python code:

```
[1] ✓ 0s
import pandas as pd

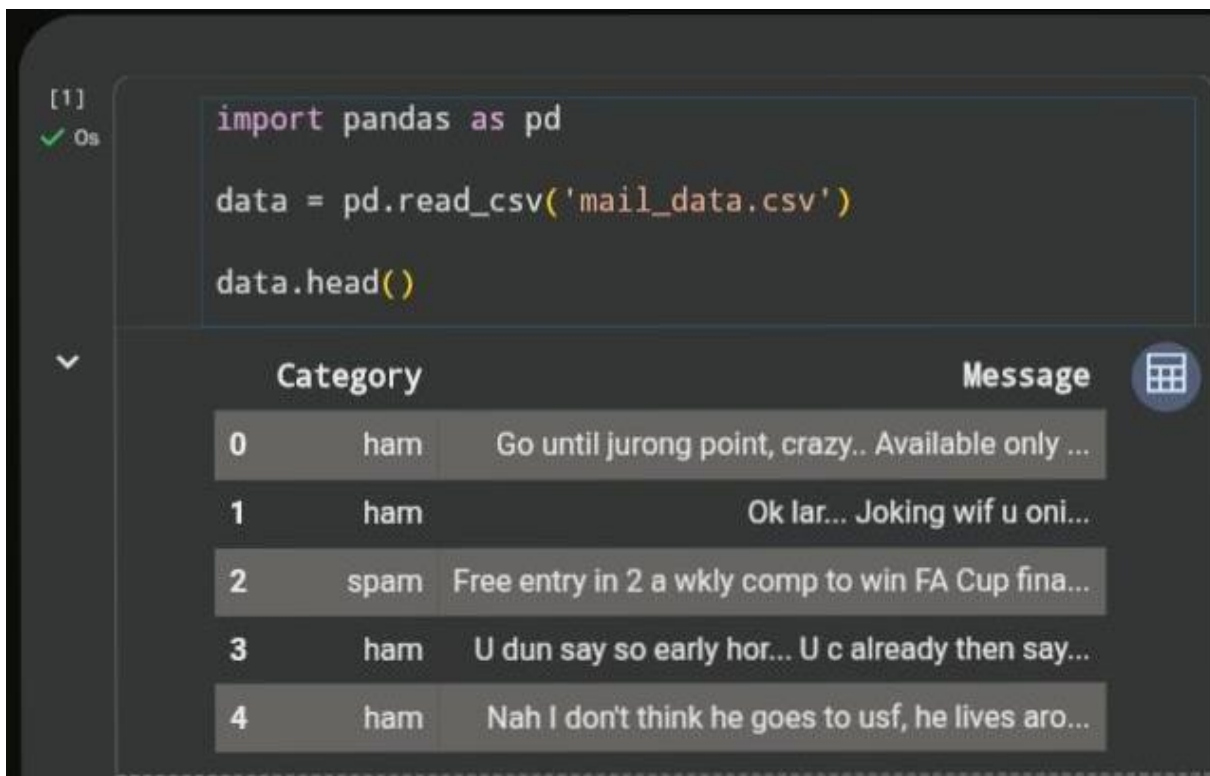
data = pd.read_csv('mail_data.csv')

data.head()
```

Below the code cell, the output is displayed as a table with two columns: 'Category' and 'Message'. The table shows the first five rows of the dataset.

|   | Category | Message                                           |
|---|----------|---------------------------------------------------|
| 0 | ham      | Go until jurong point, crazy.. Available only ... |
| 1 | ham      | Ok lar... Joking wif u oni...                     |
| 2 | spam     | Free entry in 2 a wkly comp to win FA Cup fina... |
| 3 | ham      | U dun say so early hor... U c already then say... |
| 4 | ham      | Nah I don't think he goes to usf, he lives aro... |

- Total Messages: 5572
- Categories: Spam and Ham
- Data Type: Text Dataset



This screenshot is identical to the one above, showing the same Jupyter Notebook interface with the same code cell and output table.

```
[1] ✓ 0s
import pandas as pd

data = pd.read_csv('mail_data.csv')

data.head()
```

|   | Category | Message                                           |
|---|----------|---------------------------------------------------|
| 0 | ham      | Go until jurong point, crazy.. Available only ... |
| 1 | ham      | Ok lar... Joking wif u oni...                     |
| 2 | spam     | Free entry in 2 a wkly comp to win FA Cup fina... |
| 3 | ham      | U dun say so early hor... U c already then say... |
| 4 | ham      | Nah I don't think he goes to usf, he lives aro... |

(

## 8. METHODOLOGY

The methodology followed in this project includes:

1. Data Collection
  2. Data Loading
  3. Data Preprocessing
  4. Feature Extraction
  5. Train-Test Split
  6. Model Training
  7. Model Evaluation
  8. Prediction
- 

## 9. DATA PREPROCESSING

Data preprocessing is an important step in Machine Learning.

The following preprocessing tasks were performed:

- Loading CSV dataset
- Converting labels into numerical values
- Removing unnecessary symbols
- Preparing text data for vectorization

This improves model efficiency and prediction accuracy.

```
[ ] Y = Y.astype('int')
    print(Y.head())
```

0 1  
1 1  
2 0  
3 1  
4 1  
Name: Category, dtype: int64

---

## 10. FEATURE EXTRACTION

Machine Learning algorithms require numerical data.

TF-IDF (Term Frequency-Inverse Document Frequency) was used to convert text messages into numerical vectors.

Advantages of TF-IDF:

- Removes importance of common words
  - Highlights meaningful words
  - Improves classification performance
- 

## 11. MACHINE LEARNING ALGORITHM

### Logistic Regression

Logistic Regression is a supervised machine learning algorithm used for classification tasks.

Reasons for choosing Logistic Regression:

- Fast execution
  - High accuracy
  - Easy implementation
  - Suitable for text classification
- 

## 12. MODEL TRAINING

The dataset was divided into:

- Training Data: 80%
- Testing Data: 20%

The Logistic Regression model was trained using the training dataset.

The TF-IDF transformed features were used as input.

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(X_train_features, Y_train)

from sklearn.metrics import accuracy_score
prediction = model.predict(X_test_features)

print("Accuracy Score =", accuracy_score(Y_test, prediction))
```

---

## 13. MODEL TESTING

The trained model was tested using unseen messages.

Performance evaluation was conducted using:

- Accuracy Score
- Prediction Results
- Confusion Matrix

```
[ ] input_mail = ["Congratulations! You have won a free iPhone. Click here now"]
    input_data_features = feature_extraction.transform(input_mail)
    prediction = model.predict(input_data_features)

    if prediction[0] == 0:
        print("Spam Message")
    else:
        print("Ham Message")
    ✓ Ham Message
```

---

## 14. RESULTS AND ANALYSIS

The developed model achieved excellent performance.

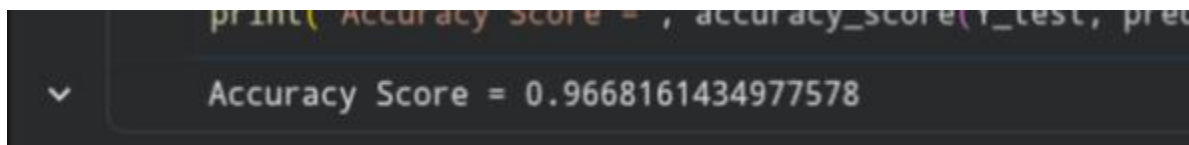
## Accuracy Score

**96.68%**

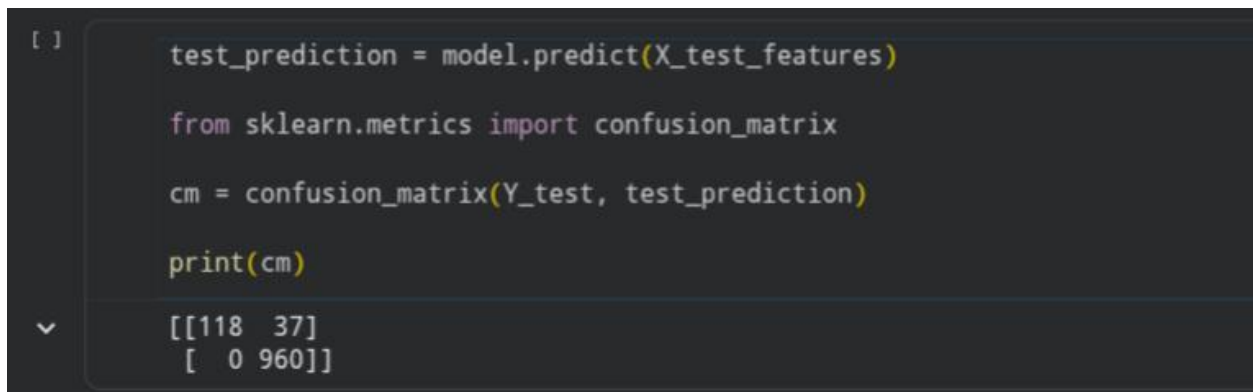
The model successfully classified spam and ham messages with high precision.

## Observations

- Most spam messages were correctly detected.
- Legitimate messages were accurately identified.
- The model demonstrated reliable classification capability.



A screenshot of a Jupyter Notebook cell. The top part shows a line of code: `print('Accuracy Score = ', accuracy_score(Y_test, pred`. The bottom part shows the output: `Accuracy Score = 0.9668161434977578`.



A screenshot of a Jupyter Notebook cell. The top part shows a block of code: `test_prediction = model.predict(X_test_features)`, `from sklearn.metrics import confusion_matrix`, `cm = confusion_matrix(Y_test, test_prediction)`, and `print(cm)`. The bottom part shows the output: `[[118 37]` and `[ 0 960]]`.

---

## 15. APPLICATIONS

The SMS Spam Detection System can be used in:

- Mobile devices
- Messaging applications
- Email spam filtering
- Online communication platforms
- Security monitoring systems

---

## 16. ADVANTAGES

- Fast message classification
- High accuracy
- Easy implementation
- Low computational cost
- Improved user security

---

## 17. LIMITATIONS

- Performance depends on training data quality.
- New spam patterns may require retraining.
- Cannot completely eliminate all spam messages.

---

## 18. FUTURE SCOPE

Future improvements may include:

- Deep Learning models
- Real-time spam detection
- Multi-language support
- Mobile application integration
- Cloud deployment

---

## 19. CONCLUSION

The SMS Spam Detection System was successfully developed using Machine Learning techniques.

TF-IDF Vectorization and Logistic Regression were applied to classify messages into spam and ham categories.

The model achieved an accuracy of **96.68%**, demonstrating its effectiveness and reliability. The project highlights how Machine Learning can automate spam detection and improve communication security.

---

## 20. REFERENCES

1. Python Documentation
  2. Scikit-Learn Documentation
  3. Google Colab Documentation
  4. SMS Spam Collection Dataset
  5. Machine Learning Research Articles
  6. Natural Language Processing Resources
- 

## APPENDIX

### Software Used

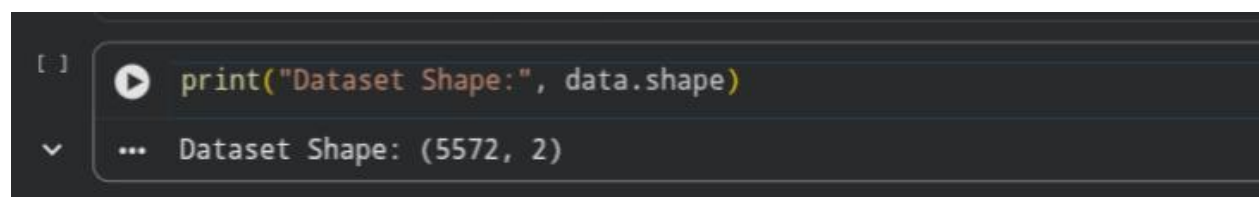
- Python
- Google Colab
- Pandas
- Scikit-Learn
- NumPy

### Hardware Used

- Android Mobile Phone
- Internet Connection

### Project Outcome

The SMS Spam Detection model successfully classified messages and achieved **96.68% accuracy**, proving the effectiveness of Machine Learning in spam detection tasks.



```
[ ] print("Dataset Shape:", data.shape)
... Dataset Shape: (5572, 2)
```

